

Fiches orales, conception et CI/CD

Utiliser comme aide-mémoire, ne pas réciter.

Phrase d'ouverture

- Je présente le projet en suivant la logique complète du développement : analyser le besoin, modéliser les interactions et les données, implémenter, tester puis déployer.

Ordre de conception à expliquer

- 1. Besoins et acteurs.
- 2. Use case UML pour les fonctionnalités par rôle.
- 3. Séquence UML pour le scénario critique.
- 4. MCD, puis MLD, puis MPD pour les données.
- 5. Architecture en couches, implémentation, tests, CI/CD et production.

MCD, MLD, MPD, version très courte

- MCD : vision métier, entités, relations et cardinalités.
- MLD : traduction relationnelle, tables logiques, PK et FK.
- MPD : implémentation SQLite avec types, contraintes, valeurs par défaut et règles de suppression.

UML, version très courte

- Use case : qui utilise quelles fonctionnalités ?
- Séquence : dans quel ordre les composants échangent-ils pendant un scénario ?
- Architecture : quelle responsabilité pour chaque couche ?

Questions probables et réponses courtes

Répondre puis illustrer immédiatement avec le diagramme ou le code associé.

Q. Pourquoi commencer par le MCD ?

R. Parce qu'il exprime le besoin métier sans subir les choix techniques. Je définis d'abord les entités, relations et cardinalités, puis je transforme en tables.

Q. Pourquoi une table FAVORIS ?

R. La relation entre utilisateurs et messages est N,N. Une table d'association avec user_id et message_id permet de la représenter proprement.

Q. Différence use case et séquence ?

R. Le use case donne la vue fonctionnelle, acteurs et services. La séquence détaille le déroulement temporel d'un scénario précis entre front, API, authentification et base.

Q. Pourquoi approved = 0 au départ ?

R. C'est la règle métier de modération. Le message existe, mais il reste invisible au public jusqu'à l'approbation administrateur.

Q. Pourquoi Turso plutôt que SQLite local sur Vercel ?

R. Vercel est serverless et son système de fichiers est éphémère. Turso fournit une SQLite hébergée persistante et compatible avec ce type d'hébergement.

Q. CI et CD, différence ?

R. La CI vérifie automatiquement le code. Le CD prépare ou effectue la livraison et le déploiement. Dans le projet, GitHub Actions contrôle, Vercel publie depuis Git.

Q. Que fait le pipeline ?

R. À chaque push ou PR, il récupère le code, installe les dépendances, exécute les contrôles de qualité puis les tests. Une erreur rend le job rouge.

Q. Quels éléments restent manuels ?

R. La revue humaine, les tests visuels, les migrations de données et la décision de pousser une version sur main.

Checklist avant soutenance

Les points à contrôler avant de présenter le projet.

Démonstration

- Préparer un compte utilisateur et un compte administrateur.
- Préparer un message de démonstration à soumettre et valider.
- Ouvrir les captures, les diagrammes et le rapport de tests comme plan B.

Cohérence des preuves

- Exécuter npm test juste avant l'oral et annoncer le nombre réellement affiché.
- Harmoniser ce chiffre dans le dossier, les diapositives et le discours.
- Vérifier que les variables Turso sont présentes dans Vercel mais absentes du dépôt Git.

Points à ne pas sur-vendre

- Dire ce qui est automatisé aujourd'hui et ce qui reste une évolution, notamment le verrouillage strict du déploiement après CI verte.
- Distinguer clairement le fonctionnement local, test et production.
- Répondre par le raisonnement et la preuve, jamais par une technologie seulement.

Sources de preuve à ouvrir si besoin

- Diagramme use case, séquence, MCD, MLD, MPD.
- Workflow GitHub Actions et journal du job.
- Vercel, variables d'environnement, scripts npm et rapport de tests.